

RWTH AACHEN UNIVERSITY
Chair of Computer Science 5 - Information Systems and Database Technology
Prof. Dr. rer. pol. Matthias Jarke

i5CloudMatch: An Entity-based Integration of Large-scale Datasets in the Cloud

SSE Master Thesis Proposal

Ammar Hamid Sahib
Matr. Nr. 306916

9.7.2013

Supervisors: Prof. Dr. Matthias Jarke, RWTH Aachen University
PD Dr. Ralf Klamma, RWTH Aachen University

Advisors: Petru Nicolaescu, M.Sc., RWTH Aachen University
Dipl.-Ing. Dejan Kovachev, RWTH Aachen University

This thesis is dedicated to
The Memory of Mathias Jaschob

Contents

Contents	3
1 Introduction	5
1.1 Motivation	6
1.2 Thesis Goals	6
2 Background	7
2.1 Entity Matching	7
2.1.1 Attribute Based Entity Matching	7
2.1.2 Relational Entity Matching	7
2.1.3 Collective Entity Matching	8
2.2 Graph-Based Entity Matching	9
2.2.1 Issues with Graph-based Entity Matching	9
2.2.2 Graph-based Similarity	10
2.3 Cloud-based Programming Models	11
2.3.1 MapReduce Programming Model	11
2.3.2 BSP Programming Model	12
2.4 Clustering Approaches	13
2.4.1 Relational Clustering	13
2.4.2 Component-based Clustering	13
2.4.3 Semi-clustering	14
2.4.4 Clustering Aggregation	14
2.5 Blocking methods	14
2.5.1 Canopies	14
2.5.2 Semantic Blocking	15
2.6 Partitioning Strategies	15
2.6.1 Size-based Partitioning	15
2.6.2 Blocking-based Partitioning	15
3 State Of The Art	16
3.1 Entity Matching Systems and Frameworks	16
3.2 Parallel/Distributed Entity Matching	19
3.3 Cloud-based Entity Matching	20
3.4 Summary	21

4	Cloud Approach	24
4.1	Simple Example	24
4.2	Theoretical Foundation	26
4.3	Proposed Solution	27
4.3.1	Load Data	28
4.3.2	Blocking	28
4.3.3	Distribution	28
4.3.4	Matching	29
4.3.5	Save Data	29
4.4	Blocking	29
4.5	Distribution	30
4.6	Matching by Collective Relational Entity Matching	30
4.7	Vertex-oriented Collective Relational Clustering Algorithm	31
4.7.1	Extending the Theoretical Foundation	35
4.7.2	Mathematical Foundation of the Communication	35
4.7.3	Complexity Analysis	36
4.7.4	Multi-type Entity Matching	36
4.7.5	Canonicalization	37
4.7.6	Optimizing for the Cloud	37
4.8	Adaptive/Incremental Entity Matching	38
4.9	Graph-based Data Warehouse	38
5	Implementation	39
5.1	Framework Architecture	39
5.2	i5Cloud Test bed	40
5.2.1	OpenStack	40
5.3	Hama	40
5.4	SimMetrics	40
6	Evaluation	41
6.1	Measuring Matching Quality	41
6.2	Measuring Framework Efficiency	42
6.3	Evaluation Plan	43
6.4	Test Data Sets	43
7	Time Plan	44
	Bibliography	45

Chapter 1

Introduction

Data integration is the process of combining data from several disparate sources, and provide a unified view of the data. One form of data integration in which information related to the same real-world entity is merged is called entity-based data integration. In entity-based data integration, information referencing the same real-world entity need to be matched, therefore we have an entity matching problem. Entity matching have applications in many domains like national census, health sector, national security, crime and fraud detection, mailing lists, online shopping, social science and bibliographic databases [Chr12].

When solving an entity matching problem, we face two challenges: *identification* problem and *disambiguation* problem. Identification problem is the problem of figuring out for each entity the set of different name references which may be used to refer to that entity [BG07]. The disambiguation problem is the problem of distinguishing references that have similar and sometimes exactly the same name and yet refer to different underlying entities [BG07].

The task of identifying and matching information that refer to the same entities within one or across several data sets is challenging for the reasons that can be highlights as following [Chr12]:

- *Lack of Unique Entity Identifiers*: Unique entity identifiers might be missing, and when available it might be subject to the lack of accuracy, completeness, robustness and consistency. If no identity identifiers are available in the data sets to be matched, then matching need to rely upon the attributes that are common across the data sets.
- *Computation Complexity*: When matching entities in a single data set, each record potentially needs to be compared with all others. When matching two or more data sets, potentially each record needs to be compared with all records in the other data sets to determine if a pair of records belong to the same entity or not. The computation complexity grows quadratically as the databases to be matched get larger.
- *Lack of Training Data and ground-truth*: In many cases, there is no ground-truth available, therefore the true status of two records that are matched across the data sets is not known.

Entity matching is an old problem that appears in many domains. It has different names in literature, it is known as *record linkage* and *record matching* in the statistics community, in database community this problem is described as *merge-purge*, *data deduplication* and *instance identification*, in the AI community, the same problem is described as *database hardening* and *name matching*, the names *conference resolution*, *identity uncertainly* and *duplicate detection*

are also commonly used to describe the same problem. We will use the term entity matching through the thesis.

1.1 Motivation

Many approaches and methods have been proposed to solve the entity matching problem, one of these methods is the collective relational entity matching. The collective relational entity matching take into consideration the relationship between entities to solve the matching problem collectively which lead to better matching quality. However, integrating large data sets using collective relational entity matching is a challenging tasks due to the complexity of the search space. The emerging of the cloud computing and its capability to process the large volume of data lead to the development of entity matching frameworks that use computational power of the cloud machines to process the entity matching tasks more efficiently. Most of the cloud-based development efforts use the MapReduce model to implement a distributed cloud-based frameworks. However for the collective relational entity matching, MapReduce is not the perfect choice since MapReduce does not excel in iterative graph-based processing.

Our idea is to develop a cloud-based large data set integration framework based on collective relational entity matching approach. We will use Pregel, a cloud-based graph-oriented model that better fits to our graph-based problem. We expect to have the good matching quality of collective relational entity matching and the efficiency of cloud-based frameworks. As far as we know, no previous work is done in this direction.

We will test our framework in the context of AERCS (Academic Event Recommender system for Computer Scientists) project¹ that is developed and currently hosted by the Chair of Information Systems². In this project, large-scale bibliographical data sets need to be integrated. Based on the integrated data from the DBLP³ and CiteSeerX⁴ data sets.

1.2 Thesis Goals

- Build a cloud-based framework that implements the different stages of the entity matching on the cloud.
- Extend the collective relational entity matching to large-scale data sets by implementing it on cloud.
- Modify the collective relational clustering algorithm to a vertex-oriented algorithm that is suitable to be executed on Pregel
- Evaluate the effectiveness and efficiency of the new algorithms using the i5Cloud as testbed and DBLP and CiteSeerX as test data sets.

¹<http://bosch.informatik.rwth-aachen.de:5080/AERCS>

²<http://dbis.rwth-aachen.de/cms>

³<http://dblp.uni-trier.de/>

⁴<http://citeseerx.ist.psu.edu/>

Chapter 2

Background

In this chapter we establish the background for our work. We start by introducing the different entity matching approaches. Then we discuss the issues with the graph-based entity matching. We also introduce the cloud-based programming models. Finally we give an overview of the different clustering, blocking and partitioning methods that are used for entity matching.

2.1 Entity Matching

The groundwork of entity matching problem was first formalized by Fellegi and Sunter [FS69]. They give the problem a probabilistic foundation as posing the entity matching problem as classification problem [RDG11]. Since then a lot of approaches proposed to solve the problem of entity matching.

2.1.1 Attribute Based Entity Matching

Traditional entity matching approaches are focused on pair-wise attribute similarities [RDG11], i.e the similarity of the attribute values of the pair to match. Elmagarmid et al. [EIV07] give an overview of the similarity functions used.

Shortcoming of this approach is that it cannot perform disambiguation, that is when having two references with the same name, they cannot be identified as separate entities when they belong to different entities [RDG11].

2.1.2 Relational Entity Matching

Alternative approaches to the probabilistic approaches are the relational approaches. Relational approaches of entity matching take structural similarity into account by looking at the attributes of related references, and incorporates them into attribute similarity score [BG04]. Relational approach avoid the complexity of the probabilistic approach by avoiding a formal probabilistic model, these approaches can handle complex relationships between different entities more easily and significantly faster as well [BG04].

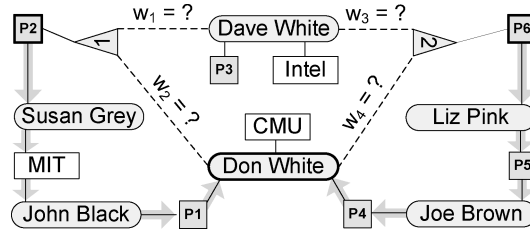


Figure 2.1: Graph for the publications example([KM06])

2.1.3 Collective Entity Matching

In traditional entity matching approaches, matching decisions are made independently, that is the decisions of matching a pair or a group of records are made independently from all other groups or records in the data set(s) that are matched or deduplicated. These approaches therefore make local decisions without taking the characteristics of all records in the data sets into account.

Bhattacharya and Geetor [BG07] introduced collective entity matching approach, which determine entities for co-occurring references jointly, rather than independently by using both attribute and relational information for determining the underlying domain entities [Chr12]. The basic idea of collective entity matching approach is to calculate the similarities of all connections in the relationship graph that are ambiguous using information from the known relationships. Since different types of entities are available, the known relationship between one type of entities can help to disambiguate of other types of entities.

In collective entity matching approach, matching decisions are aimed to be made in an overall collective fashion over all the records pairs or groups of the datasets to be matched. This approach works best when having different type of entities with certain relationships between them. These relations can be represented by a relationship graph.

Several techniques are proposed to solve the entity matching problem in collective fashion. These techniques differ in how the relationship graph is generated from the underlying datasets, and how the iterative update of the graph attributes is conducted. Collective entity matching techniques employs either iterative or hierarchical clustering, or graph-based approaches.

The collective entity matching problem can be solved in an iterative fashion. It is based on the idea that determining two references refer to the same entity may in turn allow us to make additional inferences about their related references. Using the relational evidence for collective entity matching has the potential benefit of producing more accurate matching results as well as solving both the identification and disambiguation problem at the same time. However, the potential benefit comes at cost of additional algorithmic complexity due to the propagation of the dependence between different matching decisions which results in having higher computation complexity and thus reduced scalability to large datasets.

Kalashnikov and Mehrotra [KM06] used a graph-based approach by building a relationship graph between different kind of entities that have different relations (see figure 2.1). the entity matching decisions are conducted in an iterative approach where the unknown weights are updated based on the number of connections in the path that needs to be covered to get from one end of the connection to the other.

Another graph-based approach is proposed by Dong et al. [DHM05], by constructing a dependency graph rather than a relationship graph. A node in the graph represents the similarity between a pair of entities of the same type, a connection between nodes occurs when this similarity depends upon the similarity of another pair of entities.

Bhattacharya and Geetor [BG07] proposed hierarchical clustering approach for collective entity matching. The collective entity matching is conducted using a priority queue.

Collective entity matching approaches may produce more accurate results over using only attribute similarity measures but at the cost of additional algorithmic complexity resulting from propagating the dependence between different resolution decisions [BG07].

2.2 Graph-Based Entity Matching

Graphs have natural support for representing the relationships between references. When using graphs structure to represent the references and the relationships among them, the graph has nodes for each entity reference, in case of multi-type entity references, multiple node types is used. Graph edges represent relationships between references, when having different types of relations, edges with different types can be used to represent the different types of relations that exist. Sometimes relation involves more than two references like the co-author relation, in such situation the edge is not suitable to represent relation among more than two references, instead hyper-edge is used to span all the authors who co-authorize a paper [BG06].

2.2.1 Issues with Graph-based Entity Matching

There are several interesting issues we encounter when using graphs for solving the entity matching problem. In this section we introduce some of these issues [BG06].

2.2.1.1 Multi-Type Entity Matching

In the traditional entity matching approaches, references of single-type entities are considered for the matching task. In graph-based entity matching, multiple types of entities can be solved simultaneously.

2.2.1.2 Collective Entity Matching

When the entities are related, matching decisions can be made collectively since the matching decisions depend on each other. For example in the citation domain, the relevant entities are authors, papers and venues and the relevant relationships between them are write and published in, that is an author write papers, which get published in venues. An author is a co-author of another author if they write the same paper. Having this situation, one matching decision will lead to another.

2.2.1.3 Positive and Negative Relational Evidence

Other relations may be useful to be used as relational evidence. These relations might be positive like the cited by relation when paper is cited by another paper, co-cited relation when two papers are cited by the same paper entity. If two similar paper references are co-sited by the same paper

entity, then this gives us additional evidence that they are the same. Relations may also lead to negative matching evidence like the case when two paper references are cited by the same paper entity, then it is unlikely that these two papers are duplicates.

2.2.2 Graph-based Similarity

The task of entity matching may be viewed as a graph-based clustering problem where the goal is to cluster references so that references that have identical entity labels are in the same cluster. In this section, we address the graph-based similarity measures between entity clusters based on considering the entities that they are related to via the observed edges [BG06].

2.2.2.1 Edge Detail Similarity

In graph-based clustering, we have each entity cluster associated with a set of edges to which the references contained in it belong. Recalling that each reference r is associated with a hyper-edge $r.H$, then the edge set $c.H$ for an entity cluster c may be defined as $c.H = \{r.H = h, r \in c\}$. We define a similarity measure for a pair of edges that can be used to measure the relational similarity between two clusters. What is relevant for the similarity in relational clustering are the clusters labels of the references in each edge, so for each edge, we consider the multi-set of entity labels, one for each reference associate with it. We employ the Jaccard similarity measure over these multi-set of entity labels. Let $label(h_i)$ be the set of entity labels for an adge h . Then for a pair of edges h_i and h_j , we define their similarity as:

$$sim(h_i, h_j) = \frac{|label(h_i) \cap label(h_j)|}{|label(h_i) \cup label(h_j)|}$$

Now, given this similarity measure, we can calculate the graph-based similarity between two entity clusters using an aggregation operator like *max* as follows:

$$sim_{graph}(c_i, c_j) = max_{(h_i, h_j)} \{sim(h_i, h_j)\}, h_i \in c_i.H, h_j \in c_j.H$$

Since it explicitly considers every edge associated with each entity cluster, we call this *edge detail similarity*. The computation of the *edge detail similarity* of two clusters is quadratic in the average number of edges associated with each cluster.

2.2.2.2 Neighborhood Similarity

If entity clusters have the same edge structures, that is the same neighbor cluster to which they have edges, this gives sufficient graph-based evidence that these entities should be matched. Recall that $label(l)$ is defined as the multi-set of entity labels for an edge h and for an entity cluster c , $c.H$ is the set of edges associated with it. Then the neighborhood multi-set $c.N$ for c can be formally defined as follows:

$$c.N = \bigcup_m label(h_i), h_i \in c.H$$

where $\bigcup_m$ is the multi-set union operator. Intuitively, we are looking at how many times c has participated in the same edge with another entity cluster, excluding c it self. Next, To calculate the graph-based similarity measure between two entity clusters, we calculate the Jaccard similarity between their neighborhood multi-sets.

$$sim_{graph}(c_i, c_j) = \frac{|C_i.N \cap C_j.N|}{|C_i.N \cup C_j.N|}$$

We call this *neighborhood similarity* between the two entity clusters. The computation and update of neighborhood similarity is significantly cheaper, in terms of computation, compared to edge detail similarity.

2.3 Cloud-based Programming Models

In this section we introduce the programming models suitable for the cloud-based platforms. We introduce MapReduce and Pregel programming models.

2.3.1 MapReduce Programming Model

MapReduce [DG08] is a distributed programming model that is dedicated for complex and distributed computations. It enables the parallel programming on hundreds or even thousands of shared nothing commodity PCs clusters. This programming model hides the details of the parallelism complexities and provide simple programming model consists of two functions, Map and Reduce. In the Map function the programmer specifies the computation need to be carried out in parallel fashion by the underlying hardware, the map function process the input in the form of key/value pair (which is the basic data structure of MapReduce), the computation is carried out in distributed fashion and an intermediate results are generated in the form of key/value pairs. In the Reduce function, all the intermediate values associated with the same intermediate key are merged and the final result of the computation is produced. MapReduce divides the processing into two consecutive stages: the map and the reduce phase. The mechanism works as follows: the input starts as data stored in distributed system, the mapper is applied to every key value pair to generate an arbitrary number of intermediate key/value pairs and then the reducer is applied to all values associated with the same intermediate key to generate out key/value pairs, implicit between the map and reduce is distributed group by operation in intermediate keys (see figure 2.2).

MapReduce has not been originally designed for graph-processing, However a set of good practices along with design patterns for graph processing have been developed. These good practices show how to express iterative algorithms in MapReduce, however they are helpless in overcoming the inefficiencies of MapReduce model in graphs processing [KIKK12]. The reasons of the inefficiencies of MapReduce when processing graphs are the great network overhead since the graph structure must be transferred over the network of computational nodes from Mappers to Reducers so that they can serve as an input for the next iteration. The stateless, two-phased computational nature of MapReduce does not allow vertices to reside on the same machine throughout the whole computation. After every phase (map/reduce) the entire data must be written to a global memory in order to be consumed in the next phase. Since the distributed file system serves as the global memory in the cloud environment, the whole computation is very disk-intensive. Additionally, the map and reduce workers reside on different physical machines and for that reason, the data is constantly transferred over the network, which is the scarce resource in cloud environment [KIKK12].

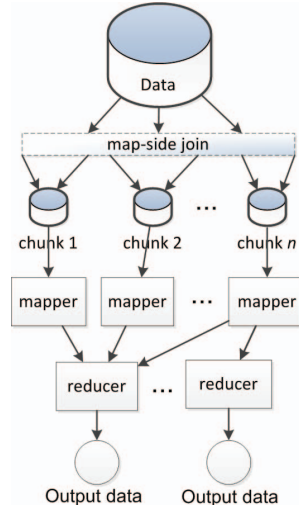


Figure 2.2: Data flow in MapReduce programming model([KIKK12])

2.3.2 BSP Programming Model

Having in mind the inefficiency of MapReduce for iterative graph processing, Google has created Pregel [MAB⁺10] - a distributed, fault-tolerant system for graph processing, based on Bulk Synchronous Parallel (BSP) processing model [Val90].

The computation in the BSP model is organized as a sequence of supersteps. Each superstep consists of two phases, the computational phase followed by the communication phase. In the computational phase, local independent computations run in parallel by the processors. In the communication phase, processors communicate data in all-to-all fashion. Between two supersteps there is a synchronization barrier that guarantees the simultaneous start of all the processors for the next step. The global synchronizations at the end of each superstep have significant advantages for large scale distributed systems since it ensures avoiding the overlapping of communication and computation phases and provide a straightforward way to store consistent partial states in checkpoints. In BSP, the computation consists of a series of supersteps. In every superstep, a user defined function is executed in parallel on every item from the dataset acting as an agent. Pregel is vertex-centric: a single agent computation has a graph representation in BSP. It consists of a node identifiers, its current value and/or state, and a list of nodes outgoing edges. In Pregel, all processing is carried out on the machines local memory. Before the computation, all vertices are partitioned and loaded into local memories of machines and remain there throughout the entire computation. In every superstep, a vertex receives messages from other vertices and executes a user defined function `compute()`, which performs local processing and sends messages to some or all of its neighbors. Once a vertex finishes the local computation, it deactivates itself and waits for all other vertices to finish. The barrier of synchronization mechanism allows the next superstep to begin when all vertices complete processing in the current superstep. Then only the vertices that have received a message are activated [KIKK12](see figure 2.3).

The BSP model has a stateful nature that facilitates efficient graph computing since graphs structure does not need to be send over the network. Only specific messages necessary for the algorithm execution are exchanged. There is no network overhead related to graph structure

passing as in the MapReduce model. Moreover, storing the whole graph structure in local memory of workers allows in-memory computation such that there should be no need of disk write-reads between iterations. However, it is possible only if the graph structure fits in memory of all workers. Otherwise, the spilling-to-disk techniques must be implemented [KIKK12].

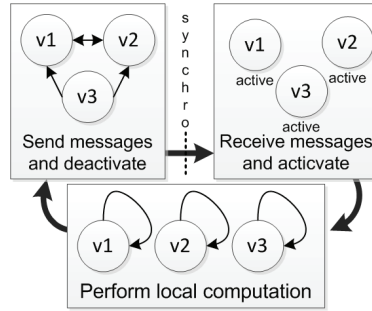


Figure 2.3: Data flow in MapReduce programming model([KIKK12])

2.4 Clustering Approaches

The task of entity matching may be viewed as a clustering problem where the goal is to cluster the references so that those that have identical entity labels are in the same cluster. The key to the success of a clustering algorithm is the similarity measure that is employed [BG06]. Different clustering methods were proposed for the entity matching problem, in this section we give an overview of these methods:

2.4.1 Relational Clustering

Bhattacharya and Getoor [BG06] Propose using greedy agglomerative clustering approach for the collective relational entity matching approach. At any stage of the clustering algorithm, each constructed entity cluster should correspond to one underlying entity and all references in that cluster should be to that entity. The greedy agglomerative clustering algorithm start by having each reference belong to separate cluster and at each step the pair of cluster that are most likely to refer to the same entity are merged.

2.4.2 Component-based Clustering

For graphs based clustering, component-based clustering can be used. Component-based clustering is based on the more general single-link hierarchical clustering method. The connected-components clustering method can be shown to be an instance of the single link method [KMPG09]. Specifically, if $d(n)$ is the distance of the two clusters to merge at step n of the single-link method, and $G(n)$ is the graph that links all data points within distance of at most $d(n)$, then the clusters after step n are the connected components of $G(n)$ [KMPG09].

2.4.3 Semi-clustering

Semi-clustering, arises in social graphs. Social graphs are analogous to the graph of references with the relationships among those references. In social graphs, vertices typically represent people and edges represent the relationships between them. edges may have weights to represent the frequency of interactions or the strength of the relationship [MAB⁺10].

A semi-cluster in a social graph is a group of people who interact frequently with each other and less frequently with others [MAB⁺10]. Semi-cluster is distinguished from ordinary clustering approaches in that vertex may belong to more than one semi-cluster [MAB⁺10]. This is similar to the canopies blocking technique where the vertex can belong to more than one canopy resulting in having overlapping canopies.

A semi-cluster c is assigned a score as following [MAB⁺10]:

$$S_c = \frac{I_c - f_B B_c}{V_c(V_c - 1)/2}$$

where I_c is the sum of the weights of all internal edges, B_c is the sum of the weights of all boundary edges (i.e., edges connecting a vertex in the semi-cluster to one outside it), V_c is the number of vertices in the semi-cluster, and f_B , the boundary edge score factor, is a user-specified parameter, usually between 0 and 1. The score is normalized, i.e., divided by the number of edges in a clique of size V_c , so that large clusters do not receive artificially high scores [MAB⁺10].

2.4.4 Clustering Aggregation

This clustering approach is particularly effective in cases when the data are defined over heterogeneous attributes that contain incomparable values. Using this approach allows us to apply the suitable clustering approach for each data set and then aggregate the individual clustering into a single clustering [GMT07].

2.5 Blocking methods

Blocking improve the efficiency of entity matching by avoiding the evaluation of the complete Cartesian product of entities. it reduces the search space for entity matching by using some kind of semantic partitioning by grouping similar entities within blocks and by restricting entity matching to entities from the same block [KKH⁺10]. In this section we give an overview of two blocking methods that are relevant to our work.

2.5.1 Canopies

Canopies [MNU00] are overlapping subsets of high-dimensional data set that is divided efficiently by using a cheap, approximate distance measures. An element may appear under more than one canopy, and every element must appear in at least one canopy. Canopies are created with the intention that points not appearing in any common canopy are far enough apart that they could not possibly be in the same cluster. Since the distance measure used to create canopies is approximate, there may not be a guarantee of this property, but by allowing canopies to overlap with each other, by choosing a large enough distance threshold, and by understanding the properties of the approximate distance measure, we can have a guarantee in some cases.

2.5.2 Semantic Blocking

Semantic blocking is new family of blocking algorithms [NMMMBLP07]. It substitutes typical blocking methods by another type of block building method that is based on the relations between the entities rather than the values of attributes the entity has. The basic idea that the semantic blocking is based on the idea that references of entities are more likely to refer to the real-life entity if they are closely related in the context established by their relations with other entities in the data set.

2.6 Partitioning Strategies

For parallel entity matching, good partitioning of data is essential to have balanced distribution of the data among the machines. Different partitioning strategies exist. One of the strategies is size-based partitioning to evaluate the Cartesian product for entity matching. The other strategy is blocking-based partitioning to combine blocking with parallel matching [KKH⁺10].

2.6.1 Size-based Partitioning

Size-based partitioning is the simplest approach and is applied for evaluating the complete Cartesian product of input entities. The goal is to split the input entities into equally-sized partitions so that each match task has to match two such partitions with each other. This approach is thus very simple and the equally sized partitions promise a good load balancing and scalability to many nodes.

2.6.2 Blocking-based Partitioning

Evaluating the Cartesian product does not scale and is thus only feasible for small-sized match problems. For larger match problems the use of blocking becomes necessary to reduce the scope for matching. Blocking entails a logical partitioning of the input entities such that all matching entities should be assigned to the same output partition, also called cluster or block. By restricting the match comparisons to the entities of the same block the match overhead can often drastically be reduced.

Chapter 3

State Of The Art

In this chapter we review state of the art of entity matching systems and frameworks. Koepcke and Rahm [KR10] comparatively analyze 11 machine learning frameworks for entity matching (see Table 3.1). We give another review of some existing systems and frameworks of entity matching [Chr12], then we give an overview of the existing cloud-based frameworks

3.1 Entity Matching Systems and Frameworks

There are several high-level requirements that should be satisfied as far as possible by entity matching systems. These requirements are:

- *Effectiveness* is the main goal of entity matching. *Effectiveness* is considered with achieving high quality results, however achieving this goal for different matching tasks typically requires combination of different match methods.
- *Efficiency* is another requirement that is important when handling large data set. The entity matching system should be fast even for large data sets. To reach good efficiency, blocking methods are usually implement as the basic techniques to reduce the search space and thus reduce the total time needed for the matching task.
- *offline/online matching* might be also a requirement to have a system that is applicable of offline and online tasks. However according to Koepcke and Rahm [KR10] all the existed frameworks are offline entity matching frameworks.
- *Genericity* refers to system's applicability of different match tasks from different data models. Most existing Entity matching systems are capable of matching relational data only. Only one framework is found to work with XML data.
- The manual efforts to employ an entity matching system should be as low as possible, particularly for selecting the matching and blocking methods and their parameters. Ideally the system should be able to handle these tasks automatically by *self-tunning* with the help of machine learning methods that utilize data training.

We give an overview from Christen [Chr12]. This overview summarize some of the entity matching systems that have mostly been developed by researchers as part of their work. Some of

these systems include graphical user interfaces, while others are more basic research prototypes with program codes only.

- BigMatch [Yan07], developed and used by the US Census Bureau, is used to match very large census data collections. Parallel versions of BigMatch have been developed to be used for large data matching tasks. The idea that is implemented in the parallel version of BigMatch is based on loading and process the smaller of the databases to the memory and represented by an inverted-index data structure. Then the indexing step is carried out by single scanning the larger database using different blocking criteria resulting into smaller size database. After that more detailed matching is carried out between the smaller database that resides in memory and the generated indexed database. The matching is conducted using a standard blocking approach. Several blocking criteria can also be applied. The program does not provide graphical user interface, rather it is controlled using text based configuration files. The input files are assumed to be flat text files with fixed record length.
- D-Dupe [BLGS06] is a graphical tool that allows interactive exploration of networks that contain duplicates. It combines data matching algorithms with network visualization to improve the efficiency of the interactive deduplication conducted by the user. D-Dupe implements many strings comparison functions like Jaro, Jacard and Monge-Elkan. In addition to that, D-Dupe considers the relational similarities in the network by calculating similarities based on the overlap of their common neighboring entities. Standard blocking is also implemented to narrow down the search of duplicates.
- DuDe [DN10] (Duplicate Detection) is a Java toolkit developed at the University of Potsdam. It contains several modules that offer application programming interfaces (API) to be used by the programmer. The first module is the preprocessor module. This module is optional and it provides statistics about the input data sets to be matched. The second module is the Algorithm module, which is the main module that contains various type of comparison functions as well as functions that aggregate similarity values. The third module is the postprocessor module that allows transitive closure calculation as well as providing various statistics. Finally DuDeOutput module provides functions to write the resulted matched data in various formats. DuDe can handle the input and the output in various formats. It can read input files of the formats of CSV files. XML documents, JSON files and bibliographies in Bibtex format. The output can be generated using the formats of CSV files and JSON files. The programmer need to program java programs that configures the data access method, the algorithms to use for indexing, attribute similarity comparisons, classification, and evaluation.
- FEBRL [Chr08] (Freely Extensible Biomedical Record Linkage) system provides techniques for improved data pre-processing, deduplication and data matching. It contains rich modules for data pre-processing, indexing, comparisons, classifications and evaluations. The data-preprocessing implements rules and hidden Markov models techniques, indexing implements 7 indexing and 8 phonetic encoding techniques, comparisons offers 26 different comparison functions and classification offers 6 different classifiers.

FEBRL comes with GUI as well as the ability for rapid development and testing of new algorithms. It can read data of various formats like CSV, TAB and COL formats with

optional modules for database access. It also have support for parallel computing that is based on the Message Passing Interface(MPI).

- FRIL [JLX⁺08] (Fine-Grained Records Integration and Linkage) is a system for schema reconciliation and data matching. It contains several indexing methods and a variety of comparison functions. It offers parameter tuning approach that is based on Expectation maximization (EM) approach. It contains GUI that allows users to select parameters such that matching fields, comparison functions, matching weights, indexing functions, and evaluation metrics to apply. Analysis window is provided to the user that allows the inspection of the effectiveness of the parameters on the matching process prior to running the actual matching on the full data sets.

FRIL supports parallelization on multi-core systems in a way that is transparent to the user, that is if FRIL detects multi-core systems then it will run algorithms in parallel.

- Merge Toolbox (MTB) [SBB04] is a java program that implements both probabilistic and distance-based deduplication and data matching. It implements a variety of comparison and German phonetic encoding functions. It also implements standards blocking techniques. For probabilistic data matching, the parameters can either be set manually or by using the EM algorithm.

The MTB can handle input files of the format of CSV, it also allows access to SATA text files. It offers the ability of privacy-preserving data matching by using the hah-encoded Bloom filters.

- OYSTER (Open YSTem Entity Resolution) [Tal11] systems offer modules for probabilistic matching, finding transitive closure of matched record sets, and indexing. It can be run in several modes to conduct operations like data matching/merge-purge, identity capture, and identity resolution. The core of the OYSTER is based on the R-Swoosh algorithm . OYSTER uses XML scripts customized by the user to define entity attributes, the table layout of the data sources and identity rules for resolving reference sources.
- RecordLinkage [SB10] is collection of scripts written in the R statistical languages. It allows the deduplication of one data set or the matching of two data sets. It includes functions for standard blocking, and several probabilistic matching approaches combined by a facility for parameter estimation using the EM algorithm. It support supervised and unsupervised classification techniques.
- SecondString toolkit [CRF03] is a set of java classes. It implements many approximate string comparison functions as well as the implementation of the SoftTFIDF and the Monge-Elkan multi-word comparison function. it also implement a combined adaptive string distance learner.
- SILK [JIB10] is a system that allows the discovery of relationships in data within the Linked data framework. SILK assumes that data to be represented as RDF (Resource Description Framework) tuples. It contains indexing and approximate string comparison functions. It uses the string comparison module of FEBRL system. SILK facilitates parallel execution using the MapReduce framework.

- SimMetrics is java Package containing large number of approximate string comparison functions. It also includes several functions that are more suitable for comparing long sequences (like genome sequences), such as the NeedlemanWunsch, Smith WatermanGotoh, and the SmithWaterman algorithms.
- Tailor (RecOrd LinkAge Toolbox) [EVE02] toolbox allows different indexing, comparison and classification techniques. it implements standard blocking and the sorted neighborhood approach. It also provides evaluation methods. It offers both clustering and classification techniques. users can interact with TAILOR either through a definition language or via GUI.
- AERCS (Academic Event Recommender system for Computer Scientists)¹ is a visualization tool for communities of academic event in Computer Science. The system is developed and currently hosted by the Chair of Information System² , RWTH Aachen University, Germany.

The system provides the functionality of browsing a list of conferences series and selecting a particular series to see the detail analysis and visualization of the community. Each series and conference has its own page which provides more detail information. Each author also has a page which list all conferences he/she has attended and a visualization of his/her research community. Users can also search conferences, series, or author by name.

Currently, co-authorship network of conferences and series from the combination of DBLP³ and CiteSeerX digital libraries dataset are visualized.

- GeRoMeSuite⁴ is a generic model management framework that enables managing models in a polymorphic fashion. It provides an environment that simplifies the implementation of model management operators like Match, Merge, and Compose. it represents the different modeling languages (such as XML Schema, OWL, SQL) in a generic way using the meta-model GeRoMe. The framework supports schema integration using simple morphisms and structured intensional mappings. GeRoMeSuite provides a workspace to manage and map models and supports importing SQL, XML Schema and OWL models.

3.2 Parallel/Distributed Entity Matching

As data sets today are getting larger, entity-based integration of them requires more computing power and storage resources. Modern Parallel/Distributed computing environments were implemented to reduce the time required to match large-scale data sets. We give an overview of some of the existing Parallel/Distributed entity matching frameworks [Chr12].

- Hernandez and Stolfo [HS95] presented a parallel version of their sorted neighborhood indexing approach. This parallel version was implemented on parallel machine where

¹<http://bosch.informatik.rwth-aachen.de:5080/AERCS>

²<http://dbis.rwth-aachen.de/cms>

³<http://dblp.uni-trier.de/>

⁴<http://dbis.rwth-aachen.de/cms/projects/GeRoMeSuite> accessed at 27.6.2013

memory was not shared between processors. the implementation is based on a master-worker approach where the master process distributes the data set, and the worker processes sort their subset of the data, and finally the master process joins the sorted subsets.

- Christen et al. [CCH04] described a system that employs the Message Passing Interface (MPI) library for communication between processors. Each processor has access to its memory and storage.
- Benjelloun et al. [BGMG⁺] proposed D-Swoosh algorithms that facilitate distributed data matching. The algorithms are based on Swoosh algorithm. Each processor runs the R-Swoosh algorithm on its subset of the data, and each matched set of records is communicated to all processors that hold a record that is affected by a merge.
- Kawai et al. [KGMB⁺06] implemented P-Swoosh algorithm that is based on master-worker approach of D-Swoosh. They explored different ways of how to distribute matching record sets. Two different load balancing techniques were investigated, one with a static distribution and the other with an adaptive distribution.
- Kim and Lee [KL07] investigated different approaches to match and merge data sets that are either clean (i.e. the two source data sets do not contain duplicate records) or dirty (i.e. the source data sets can contain duplicate records). The general idea of their approach is that the smaller of the two data sets to be matched, is replicated on each processor. The larger data set, is however distributed.
- Kirsten et al. [KKH⁺10] proposed different strategies of how to distribute data to generate independent matching tasks that can be executed on different processors. The proposed strategies aims to set the number of partitions and their sizes in an optimal way with regard to both communication overhead and memory requirements.

3.3 Cloud-based Entity Matching

In this section we give a review of the existing cloud-based entity matching frameworks. All of the reviewed frameworks are using the MapReduce programming model for distribution of the tasks.

- Dal Bianco et al. [DBGH11] proposed a data parallelism approach for multi-core computing platforms based on the MapReduce programming model. They proposed an efficient indexing approach based on standard blocking that, similar to the approach presented by Kirsten et al. above.
- Kolb et al. [KTR12] proposed a similar approach to parallel data matching also based on the MapReduce programming model. They developed two efficient parallel variations of the sorted neighborhood indexing technique. In the first approach, multiple Map-Reduce processes are employed, while the second approach employed a tailored data replication to improve performance. A pre-processing step was applied on the data to be matched that calculates a matrix that contains information about how many entities are given per blocking key. Based on this information an efficient load balancing of entities onto processors can be achieved.

- Rastogi et al. [RDG11] proposed a framework to improve the scalability of collective matching techniques by running a collective matching process many times on small subsets of data set that are in the same neighborhood of the data. These independent collective matching instances exchange messages about the local matches found, and the results of all matching instances are combined into a final overall solution. They reported the use of MapReduce to achieve parallelism.
- Kolb et al. [KKTR11] investigates how learning based entity resolution can be realized in a cloud infrastructure using MapReduce. They proposed and evaluated two MapReduce-based strategies for pair-wise similarity computation and classifier application on the Cartesian product of two input sources.
- Kolb et al. [KTR12] introduced Dedoop, a MapReduce-based entity matching framework for large data sets. Dedoop supports a browser-based specification of complex work flows including blocking and matching steps as well as the optional use of machine learning for the automatic generation of match classifiers.

3.4 Summary

In this chapter we reviewed a lot of frameworks that use different approaches. Many frameworks use machine learning techniques with data training to solve the entity matching problem (see Table 3.1). We note in Table 3.1 that no system satisfies all the requirements. In fact some of the posed requirements are in conflict with each other, for example the use of blocking methods improves efficiency but effects the effectiveness, combining several matching methods may improve the effectiveness but it effects the efficiency. In addition to that, these systems can not scale well for large data sets.

In Table 3.2 we can see other entity matching frameworks. Some of them are systems with graphical user interface and parameter tuning, and some others are just libraries with no graphical user interface. just half of the frameworks have support for the parallel/distributed entity matching. Database access is rarely supported by these frameworks and only few of them provide application programming interface (API).

We also reviewed some parallel/distributed entity matching frameworks. Some of them use message passing interface (MPI) and others use the MapReduce programming model to implement the framework on the cloud. However we were unable to find cloud-based frameworks that use the bulk synchronous parallel (BSP) programming model and its graph-based Pregel programming model.

In this thesis work, we develop a cloud-based entity matching framework based on the collective relational entity matching approach. We view the entity matching problem as graph-based clustering problem and use Pregel programming model to implement the graph-based clustering algorithm. To the best of our knowledge no previous work was done with such direction. Our framework enhance both the Effectiveness and Efficiency in terms of quality and scalability. In addition to that, our framework provides graphical user interface, take into consideration the relations between entities, supports different input formats like CSV and XML, supports database access, provides application programming interface (API) and supports both merging and deduplication.

Table 3.1: Overview of machine learning entity matching frameworks(constructed from [KR10]

System	Training based	Entity type	Disjoint partitioning	overlapping partitioning	Matchers	Matcher combination	Learners	Training selection
BN [LCW07]		XML			Attribute, Context	Numerical		
MOMA [TR]		Relational			Attribute, Context	Workflow		
SERF [BGMM ⁺ 09]		Relational			Attribute	Rules		
Active Atlas [TKM02]	Yes	Relational		Hashing	Attribute	Rules	Decision tree	Manual, semi-automatic
MARLIN [BM03]	Yes	Relational		Canopy	Attribute	Numerical, rules	SVM, decision tree	Manual, semi-automatic
Multiple Classifier System [ZR05]	Yes	Relational	?	?		Numerical, rules	7 (SVM, decision tree, etc.)	Manual
Operator [CCGK07]	Yes	Relational		Canopy	Attribute	Rules	Operator Tree algorithm, SVM	Manual Manual
TAILOR [EVE02]	Yes	Relational	sorting, hashing	sorted neighborhood	Attribute	Numerical, rules	Probabilistic, decision tree	Manual
FEBRL [Chr08]	Yes	Relational	Threshold	neighborhood, q-gram, Canopy	Attribute	Numerical	SVM	Manual, automatic
STEM [KR08]	Yes	Relational		sorted neighborhood	Attribute	Numerical, rules	SVM, decision tree, learning logistic regression, multiple	Manual, semi-automatic
Context Based Framework [CKM09]	Yes	Relational		Canopy	Attribute Context	Numerical, rules	Diverse (SMOreg, Logistic, etc.)	Manual

Table 3.2: Overview of entity matching systems and frameworks

System	Parallel/ Distributed	GUI	Relational	Input Format	API	Database Access	Parameter tuning	Deduplication	Merging
BigMatch [Yan07]	✓	✗	✗	flat file	✗	✗	✗	?	?
D-Dupe [BLGS06]	✗	✓	✓	?	✗	✗	✗	?	?
DuDe [DN10]	✗	✗	✗	XML, CSV, JSON	✓	✗	✗	?	?
FEBRL [Chr08]	✓	✓	✗	CSV, TAB, COL	✓	✓	✗	?	?
FRIL [JLX ⁺ 08]	✓	✓	✗	?	✗	✗	✓	?	?
Merge toolbox [SBB04]	✗	✓	✗	CSV	✗	✗	✓	?	?
OYSTER [Tal11]	✗	✗	✗	?	✗	✗	✗	?	?
R Record Linkage [SB10]	✗	?	✗	?	✗	✗	✓	✓	✓
SecondString [CRF03]	✗	✗	✗	?	✗	✗	✗	?	?
SILK [JIB10]	✓	✗	✓	RDF	✗	✗	✗	?	?
Simmetrics	✗	✗	✗	?	✗	✗	✗	?	?
TAILOR [EVE02]	✗	✓	✗	?	✗	✗	✗	?	?
AERCS	✗	✓	✓	CSV	✗	✗	✗	?	✓
i5CloudMatch	✓	✗	✓	CSV, XML	✓	✓	✗	✓	✓

Chapter 4

Cloud Approach

In this chapter we introduce the collective entity matching problem and give our solution to implement it on the cloud. We start by giving a simple example, theoretical foundation, and then we explain the different components of our solution.

4.1 Simple Example

As an example we use a bibliographical example presented by Bhattacharya and Getoor [BG06]. In this example we consider the problem of resolving the authors in academic publications databases like DBLP and CiteSeer. Consider the following set of four papers:

1. A. V. Aho, S. C. Johnson, J. D. Ullman, "code generation for machines with multi-register operations"
2. A. V. Aho, J. D. Ullman, "The university of database languages"
3. A. V. Aho, J. D. Ullman, "Optical partial-match retrieval when fields are independently specified"
4. Alfredo V. Aho, Jeffrey D. Ullman, Scott Johnson, "Code generation for expressions with common sub-expressions"

We would like to find out, given these papers, which of the referenced author names belong to the same author entities. For instance we need to determine whether paper 1 and paper 2 are written by the same author named Ullman, or whether the name references belong to different authors. The same need to be considered for the Ullman from paper 3 and Ullman from paper 4, and all pairwise combinations. Similar questions about the author names Johnson and Aho also need to be considered in this context.

In order to resolve these authors references, we may begin by examining the author references to see which ones we consider to be the same. In the first step we might decide that all of the Aho references refer to the same author entity because Aho is an unusual last name. This corresponds to resolving all the Aho references into a single entity. However, we might be not quite sure that the Ullman references are the same author, and we certainly not sure about Johnson references, since Johnson is very common name. But deciding that the Aho references

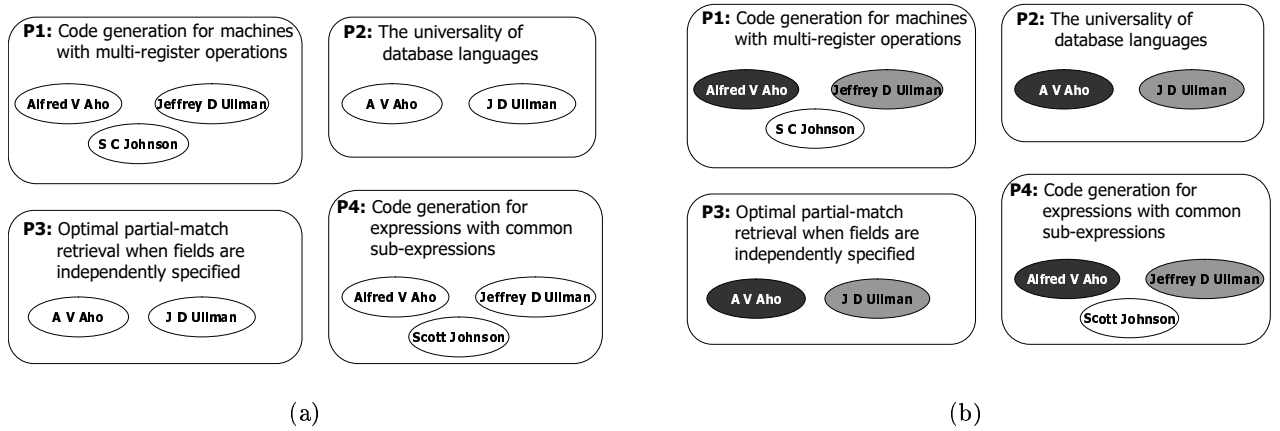


Figure 4.1: (a) An example author/paper resolution problem. Each box represents a paper reference and each oval represents an author reference. (b) The resolved entities corresponding to the example author/paper resolution problem. ([BG06])

correspond to the same entity gives us additional information for the Ullman references. We know that the references share a common co-author. Now with a higher confidence we can consolidate the Ullman references. Based solely on the Aho entity consolidation, we do not have enough evidence to consolidate the Johnson references. However, after making the Ullman consolidations, we may decide that having two co-authors in common is enough evidence to tip the balance, and that all the Johnson references correspond to the same Johnson entity Figure 4.1.b. We end up having three author entities. We will call these entities *Alfred V Aho*, *Jeffrey D Ullman*, and *Scott Johnson*. This is illustrated by figure 4.1(b), where references correspond to the same authors are shaded identically.

In this example we can find instances of an identification problem. The identification problem can be seen in the references with different names which refer to the same entity. For example entity author 'Jeffery David Ullman' might be referenced as 'J. D. Ullman', 'Jeffery D. Ullman', 'J. Ullman' and so on. another problem we might find is the disambiguation problem which can be seen in references that have the exact same names but refer to different entities.

To solve the identification and disambiguation problems, relationships between references can be used to better resolve the references to their corresponding real entities [BG07]. These relationships can be represented as a graph where nodes represent the references and the hyper-edges represent the relationships between references. In our example the graph can be constructed such that the nodes represent the authors and the hyper-edges represent the co-authorship relation that hold between them. Figure 4.2(a) shows the reference graph for our bibliographic example. Given this graph representation for our data, our goal is to take into consideration the hyper-edges to better partition the references into entities. This can gives better results because references with similar or exact names are more likely to be the same entity if their hyper-edges connect to the same entities as well. We can see this in our example (figure 4.1), first observe that the Ullman references collaborate with Aho's who correspond to the same author. This make it more likely that they are the same entity. From the relationship graph between the

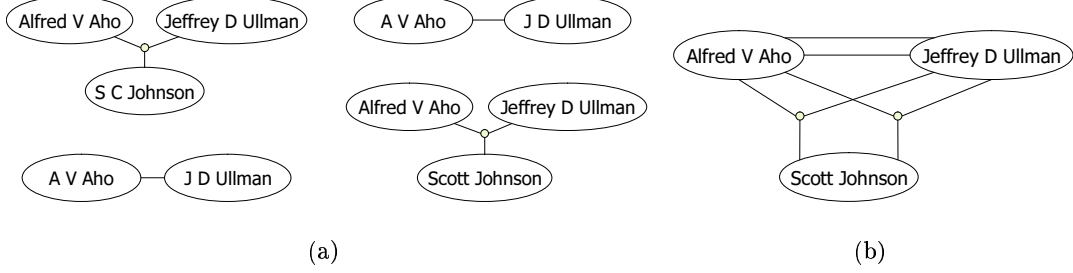


Figure 4.2: (a) The reference graph and (b) the entity graph for the author resolution example. ([BG07])

references we incrementally constructs the entity graph by considering as evidence the entity relationships that it has already discovered earlier. Figure 4.2(b) shows the resulting entity graph for our example after all the references have been correctly resolved.

4.2 Theoretical Foundation

In this section, we introduce the notation to be used for describing the relational entity matching problem based on the work of Bhattacharya and Getoor [BG07].

To formalize the problem we need notations for references, entities and hyper-edges. In the entity resolution problem, we are given a set of references $\mathcal{R} = \{r_i\}$, where each reference r has attributes $r.A_1, r.A_2, \dots, r.A_k$. The references correspond to some set of unknown entities $\mathcal{E} = \{e_i\}$. We introduce the notation $r.E$ to refer to the entity to which reference r corresponds.

The problem is to recover the hidden set of entities $\mathcal{E} = \{e_i\}$ and the entity labels $r.E$ for individual references given the observed attributes of the references. In the context of collective entity matching, we assume that the references are not observed independently, but that they co-occur. We describe the co-occurrence with a set of hyper-edges $\mathcal{H} = \{h_i\}$. Each hyper-edge h may have attributes as well, which we denote $h.A_1, h.A_2, \dots, h.A_l$, and we use $h.R$ to denote the set of references that it connects. A reference r can belong to zero or more hyper-edges and we use $r.H$ to denote the set of hyper-edges in which r participates. Considering again our example, we can see how this notation is represented.

Lets show how this applies to our example, Figure 4.3(a) shows the references and hyper-edges. Since each author name corresponds to a reference, then there are ten references r_1 through r_{10} . In this case, the attributes are the names themselves, so for example $r_1.A$ is "Alfred Aho", $r_2.A$ is "S. C. Johnson" and $r_3.A$ is "Jeffrey Ullman". Now considering the entities, we have a set of true entities as shown in figure 4.3(b). where $e_1.A =$ "Alfred V. Aho", $e_2.A =$ "S. C. Johnson" and $e_3.A =$ "Jeffrey D. Ullman". We observe that references r_1, r_4, r_6 and r_8 correspond to "Alfred V. Aho", so that $r_1.E = r_4.E = r_6.E = r_8.E = e_1$. In similar way, we observe that $r_3.E = r_5.E = r_7.E = r_{10}.E = e_2$ and $r_2.E = r_9.E = e_3$. Now considering the Hyper-edges, we have hyper-edges $\mathcal{H} = \{h_1, h_2, h_3, h_4\}$, one for each paper. The attributes of the hyper-edges in this domain are the paper titles; for example, $h_1.A_1 =$ "Code generation for machines with multi-register operations". The references r_1 through r_3 are associated with

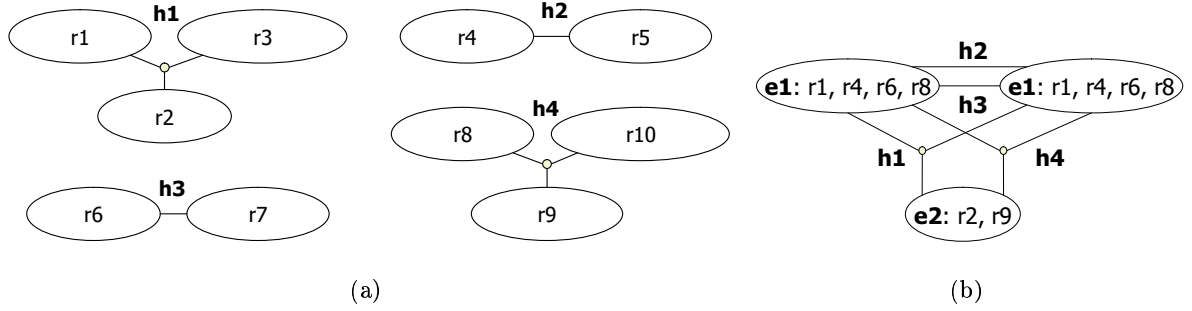


Figure 4.3: (a) A more abstract representation of the reference graph for the author resolution example; the r 's are references and the h 's are hyper-edges. (b) An abstract representation for the entity graph for the author resolution example; the nodes are the entities, the set of references they correspond to are listed, and the h 's are hyper-edges.. ([BG07])

hyper-edge h_1 , since they are the observed author references in the first paper. This is represented as $h_1.R = \{r_1, r_2, r_3\}$. We similarly represent the hyper-edge associations of the other references. The problem here is to figure out from the attributes $r.A$ and the hyper-edge label $r.h$ of the references that there are three distinct entities such that r_1, r_4, r_6 and r_8 corresponds to one entity, r_3, r_5, r_7 and r_{10} correspond to the second entity and r_2 and r_9 correspond to the third. Here we have introduced the notation assuming that all references correspond to the same class of entities, It may be extended to handle multiple entity classes and multiple types of hyper-edges between references.

4.3 Proposed Solution

In this section we describe the solution approach . The solution approach consists of several stages as shown in figure 4.4. We give an overview of each stage and discuss the problems need to be solved in each stage.

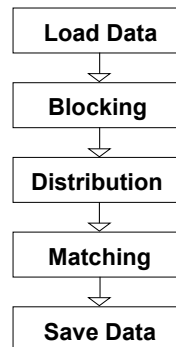


Figure 4.4: Block Diagram of the Solution Approach

4.3.1 Load Data

In load data stage. Data from different sources having different formats need to be loaded into the framework.

4.3.2 Blocking

In this stage, the loaded data need to be blocked such that similar entities are grouped into blocks. The blocks may overlap with each other. Figure 4.5 shows the references to authors grouped into blocks.

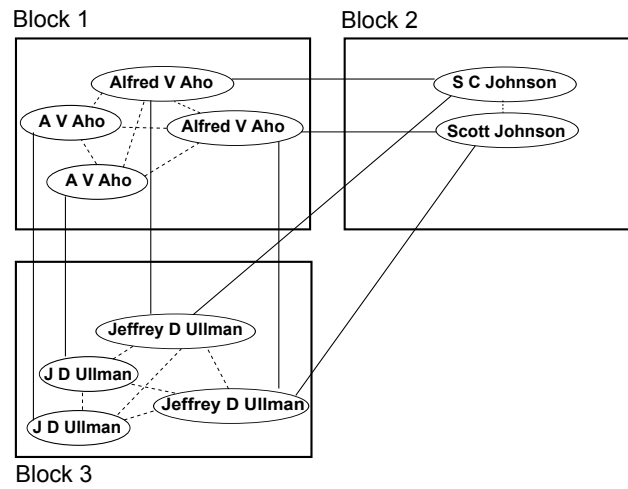


Figure 4.5: Blocking of the reference graph (solid: links to neighbor references, dashed: links to references in the same block)

4.3.3 Distribution

After the data records are grouped to overlapped blocks (canopies), we need to distribute the data over the machines of the cloud. the distribution should be balanced and should also ensure that overlapped blocks reside in the same machine. To illustrate that we added more author references to our running example. The new author references are grouped in *block4* and *block5*. *block5* overlaps with both *block2* and *block4*. The blocks are distributed among two machines such that overlapped blocks resides in single machine and the blocks distribution among the two machines is balanced as much as possible (see figure 4.6).

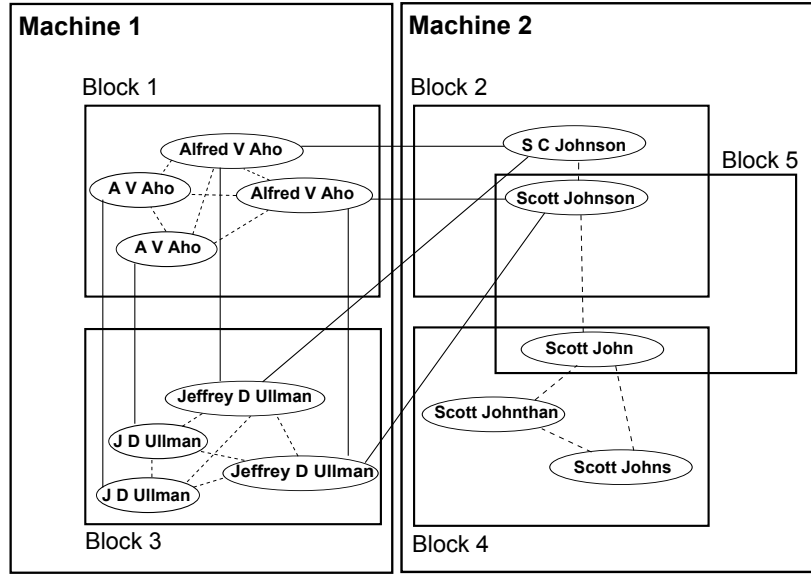


Figure 4.6: Overlapped blocks should reside in single machine

4.3.4 Matching

In this stage, the blocked data records that have similarity measure above the user defined threshold will be merged. We use the Collective relational entity matching approach and implement it on Pregel.

4.3.5 Save Data

After the data is matched, we need to store the results. statistical and log files should also be stored.

4.4 Blocking

For data blocking, we implement the meta-blocking general procedure proposed by Papadakis et al. [?] to create and process the blocks. In meta-blocking, The initial set of blocks is transformed into a new one with substantially fewer comparisons and equally high effectiveness. The new set of blocks is constructed from the most similar pairs of entities. The procedure starts by building an abstract graph representation of the original set of blocks. In this graph, the nodes correspond to entity references and the edges connect the co-occurring ones. During the creation of this structure all redundant comparisons are discarded, while the superfluous ones can be removed by pruning of the edges with the lowest weight. This blocking procedure is built upon an idea from redundancy-positive blocking techniques. In redundancy-positive blocking techniques, the number of common blocks between a pair of entity profiles is proportional to their similarity and, thus, the likelihood that they are matching. So the more blocks two entity references share, the more likely they are to describe the same real-world entity. Its interesting to notice that

this is a contrary to the redundancy-negative Canopy clustering technique used in the original Collective entity matching approach [?].

The efficiency of redundancy-positive blocking methods can be improved by discarding the unnecessary comparisons of their blocks. In general, comparisons of this kind are distinguished into two categories: (i) the redundant ones, which repeatedly compare the same entities across different blocks, and (ii) the superfluous ones, which involve non-matching entities [?].

We can mainly enhance the efficiency of redundancy-positive blocking methods by operating at the coarse level of entire blocks. We use Block Purging [?] to a-priori discards *oversized blocks*, which involve an excessively high number of unnecessary comparisons. A similar technique is Block Pruning [?]. This technique assumes an ordered set of blocks and terminates their processing as soon as duplicate overhead (i.e., the cost of identifying new duplicates) exceeds a predefined threshold [?].

Meta-blocking operates efficiently because it skips the high complexity of computing pairwise, string-based entity similarities, relying instead on the block-to-entity profile associations of the input set of blocks [?].

4.5 Distribution

Good partitioning of input data is a key prerequisite for parallel entity matching since it influences processor utilization, communication overhead, and load balancing [KKH⁺10]. Since our algorithm is communication intensive, having good data partitioning is critical to have a local communication overhead centered in the machines of the cloud such that networking overhead is minimized.

To partition the input data we use a block-based partitioning approach proposed by Kirsten et. al. [KKH⁺10]. This approach is called *partition tuning* and it split or combine blocks as following:

- We start by determining all large blocks for which the memory requirements exceed the maximum allowed size. We split these block into several equally sized partitions that obey the size restriction.
- Then we aggregate all smaller blocks that have size below some fraction of the maximal partition size into larger ones.

4.6 Matching by Collective Relational Entity Matching

Our solution approach is based on the collective relational entity matching introduced by Bhattacharya and Getoor [BG07]. The main key idea of collective relational entity resolution, is that resolutions are not made independently, but instead one resolution decision affects other resolutions via hyper-edges [BG06].

In this approach, a relationship graph is built where references are represented as nodes and relationships between these references are represented as edges between corresponding nodes. The edges can be between more than two nodes (called hyper-edges). The similarity between two nodes is calculated as the weighted sum between attribute value similarity and the relational similarity. The relational similarity is calculated based on the connectivity of the two nodes

through their hyper-edges as well as the connectivity of the neighboring nodes they are connected to. For collective entity resolution, we define the similarity of two clusters c_i and c_j as [BG06]:

$$sim(c_i, c_j) = (1 - \alpha) \times sim_A(c_i, c_j) + \alpha \times sim_R(c_i, c_j), 0 \leq \alpha \leq 1$$

where $sim_A()$ is the similarity of attributes and $sim_R()$ is the relational similarity between references in the two entity clusters. The relational similarity $sim_R()$ is dynamic in nature, which is one of the most important and interesting aspects of the collective approach [BG07]. Worth to notice here that for collective resolution, the similarity of two clusters depends on the current cluster labels of their neighbors, and therefore changes as their labels are updated.

For collective entity resolution, relational similarity considers the cluster labels of all these connected references. Recall that each reference r is associated with one or more hyper-edges in $\mathcal{H}$. Therefore, the set of hyper-edges $c.H$ that we need to consider for an entity cluster c is defined as [BG07]:

$$c.H = \bigcup_{r \in R \wedge r.C=c} \{h | h \in \mathcal{H} \wedge r \in h.R\}$$

Cluster c is connected to other clusters by these hyper-edges. For any cluster c , the set of other clusters to which c is connected via its hyper-edge set $c.H$ form the neighborhood $Nbr(c)$ of cluster c [BG07]:

$$Nbr(c) = \bigcup_{h \in c.H, r \in h.R} \{c_j | c_j = r.C\}$$

4.7 Vertex-oriented Collective Relational Clustering Algorithm

As we already seen, Bhattacharya and Lise [BG07] introduced an algorithm for relational clustering (see Figure 4.7). They used greedy agglomerative algorithm that finds the closest cluster pair at each step and merge them. On the other hand, Rastogi et. al. [RDG11] proposed a principled framework to scale any generic entity matching algorithm by running a collective matching process many times on small subsets of records that are in the same neighborhood of the data. These independent collective matching instances exchange messages about the local matches found, and the results of all matching instances are combined into a final overall solution [Chr12]. They reported an implementation of the framework on MapReduce.

We combine the collective relational entity matching approach with the ideas of scalable distributed entity matching with message passing to develop a cloud-based framework and implement it using Pregel programming model. Figure 4.8 gives high level description of our algorithm that is written to be vertex-oriented.

```

1   Find similar references using blocking
2   Initialize clusters using bootstrapping
3   For clusters  $c_i, c_j$  such that  $\text{similar}(c_i, c_j)$ 
4       Insert  $\langle \text{sim}(c_i, c_j), c_i, c_j \rangle$  into priority queue
5   While priority queue not empty
6       Extract  $\langle \text{sim}(c_i, c_j), c_i, c_j \rangle$  from queue
7       If  $\langle \text{sim}(c_i, c_j) \rangle$  less than threshold, then stop
8       Merge  $c_i$  and  $c_j$  to new cluster  $c_{ij}$ 
9       Remove entries for  $c_i$  and  $c_j$  from queue
10      For each cluster  $c_k$  such that  $\text{similar}(c_{ij}, c_k)$ 
11          Insert  $\langle \text{sim}(c_{ij}, c_k), c_{ij}, c_k \rangle$  into queue
12      For each cluster  $c_n$  neighbor of  $c_{ij}$ 
13          For  $c_k$  such that  $\text{similar}(c_k, c_n)$ 
14              Update  $\text{sim}(c_k, c_n)$  in queue

```

Figure 4.7: High-level description of the relational clustering algorithm by Bhattacharya and Lise [BG07]

```

1   Find similar references using blocking
2   Build the reference graph
3   For each block of references
4       Add an edge between each two nodes in the block
5   Initialize clustering, each cluster contains single reference
6   For each block do
7       Find the two clusters with the highest similarity
8       If the highest similarity is above user threshold
9           Merge the clusters that have highest similarity
10          Notify all neighbor clusters of the new merged cluster
11      Else
12          Exit
13      End If
14  For each cluster that is notified from step 10
15      Re-calculate similarity between references of the cluster block
16  Go to step 6

```

Figure 4.8: High-level description of the vertex-oriented relational clustering algorithm

In our algorithm, we implement the relational clustering algorithm of collective entity matching introduced by Bhattacharya and Lise [BG07]. However, since this algorithm is not suitable for a cloud computing platform like Pregel, we need to re-write the relational clustering algorithm considering Pregel’s vertex-oriented model of computing. In Pregel, programs are expressed as a sequence of iterations. In each iteration, a vertex can, independently of other vertices, receive messages sent to it in the previous iteration, send messages to other vertices, modify its own and

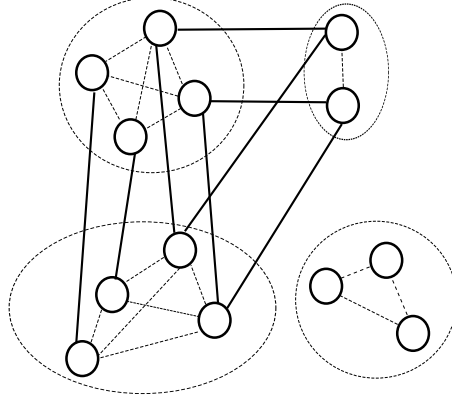


Figure 4.9: References graph (solid-lines: relation edge , dashed-line: block edge)

its outgoing edges states, and mutate the graph's topology. In other words, we need to think like a vertex.

Next we explain the algorithm steps in more details. For the implementation on cloud, we use MapReduce for step 1, and Pregel like system for the rest of steps.

In step 1, references are assigned to buckets using an attribute similarity measure. Each bucket has a representative reference that is the most similar to all references currently in the bucket. A naive implementation algorithm yields a $O(n(b + f))$ for n references and b buckets and when a reference is assigned to at most f buckets. This can be improved by maintaining an inverted index over buckets. For example, when dealing with names, for each character we maintain the list of buckets storing last names starting with that character. Then the buckets can be looked up in constant time for each reference leading to an $O(nf)$ algorithm [BG07].

In step 2, references graph is built (see Figure 4.9). In this graph, the references nodes are connected by two types of edges. The first type is relation edges (the solid line in the figure) which results in having complete graph (Every two vertices are connected) with no-self loop. The second type of the edges connects every two nodes within each block (the dashed lines in the figure) which results in having complete graph with self loop. We need to add the block edges because nodes within each block need to exchange messages during the entity matching process, in Pregel, message exchange between nodes is possible only through edges.

In step 3, the clustering is initialized by simply putting each reference node in single node cluster such that every reference has its own cluster. This is different from the original relational clustering algorithm where relational bootstrapping is necessary. In the initialization step, two lists are initialized in each node. The first list, we call the neighborhood list, contains information of the relational neighbor clusters. The second list, we call block list, contains information of the clusters that are in the same block that the node is.

In step 4, the hierarchical agglomerative clustering start in each block. In each block, the two clusters having the highest similarity between them are searched. The search is restricted within each block. If the highest similarity is above the user-specified threshold, the two clusters are merged. upon merging the two clusters, all the relational neighbors (connected by relational edges) are notified. We use the block edges to store the values of the similarity between the corresponding clusters. When the cluster (represented by vertex) need to find its highest similarity

value it need to iterate through the list of its outgoing edges and choose the highest value. This is another difference to the original collective clustering algorithm that uses a priority queue to order the clusters merge. When implementing the hierarchical agglomerative clustering in Pregel model, we need to follow the following steps:

- Finding the two clusters that have the highest similarity measure requires that each node communicates to all the other nodes within the same block. The method follows the following supersteps:
 - In the first superstep, each node sends its attribute information as well as the information of its relational neighbor clusters to all other nodes in the block (by querying the block list). These information is used by the target node to compute the attribute and the relational similarity such that each node compute the similarity between it self and every other node in the block.
 - In the second superstep, each node compute its similarity against all the other nodes and propagate the highest similarity measure value computed by itself. When the other nodes receives this similarity measure value, they compare it to the values of similarity measure computed by them selves. If the receiving node finds that highest similarity value that is received from all the nodes, if this value exists within its computation then the node recognize that it has the highest similarity between it self and another cluster within the block, and they need to merge in this iteration. otherwise the node does nothing in this iteration and waits for the next iteration to check if the next highest similarity is within its computation.
 - In the third superstep, the two clusters with the highest similarity between them merges into new cluster. This requires unifying the cluster label, In addition, some extra work need to be done:
 - * The neighborhood list of the new cluster need to be constructed by concatenating the neighborhood lists of the two merged clusters.
 - * The block list of the new cluster need to be constructed by copying the block list from one of the merged clusters
 - * The other clusters in the block need to update their block list by removing the entries of the old merged clusters and adding the entry of the new cluster
 - * The relational neighbor clusters need to update their neighbor lists by removing the entries of the old merged clusters and adding the entry of the new cluster
 - * Since the merge of clusters changes the relational structure of the reference graph, this effects the relational similarity, therefore the computation within all the blocks that include one of the relational neighbor clusters of the new cluster need to be done again. In fact this step gives the collective feature for the entity matching process.

Data partitioning is very important issue for the efficiency of our algorithm. We should aim for reducing the network overhead by having the blocks, each located completely in the same machine. Ideally, By assuring that data block are not distributed in many machines, we can have processing locally centered in each machine with minimum network traffic that is needed only when merging clusters since their neighbor clusters that need to be notified reside in other machines.

4.7.1 Extending the Theoretical Foundation

Now we are in good position to extend the theoretical foundation of the collective entity matching in order to be able to describe our vertex-oriented implementation of the collective entity matching approach. We start by having each reference in single element cluster, we use C to denote the set of clusters $\mathcal{C} = c_i$. We map each single-element cluster c_i to a vertex v_i in the graph $\mathcal{G}$ such that we have one-to-one mapping between single-element clusters and the vertices in the graph. Instead of the hyper-edges, we use bidirectional edges $E = e_i$. Each edge can hold one value. This value is used to hold different values depending whether the edge is used to represent the relations between vertices (i.e. the references) or it is used to connect the vertices in the same block (i.e. similar references).

Each vertex needs to send messages to the right vertices only. In order for the vertex to be able to do that, it needs to store and maintains four different lists. Considering that each vertex represents a cluster of references, we introduce the following notations for each cluster c :

- $c.B$: To denote the list of blocks that c belongs to, we need this list since we use canopy blocking (i.e. overlapped blocks).
- $c.BC$: To denote the list of block clusters, i.e. the clusters in the same blocks that c belongs to. The vertex needs to send messages to clusters in this list when comparing references for matching.
- $c.NC$: To denote the list of relational neighbor clusters, i.e the clusters that are connected to c by relational edges. The vertex need to send messages to the clusters in this list only when it merges with another cluster.
- $c.MC$: To denote the list of merged clusters, i.e. clusters that are already merged with c . The clusters in this list need not to receive messages from c .

The original collective relational clustering algorithm uses a priority queue to coordinate the priority of clusters merging in the agglomerative hierarchical clustering. In our vertex-oriented collective relational clustering, we use the value attribute of edges that connect vertices in the same block to store the similarity value between the cluster and the other clusters that the edges towards to instead of using a single priority queue. The vertex (i.e. cluster) needs to iterate through its outgoing edges in order to get its similarity with all other clusters in the block. This solution have many desired benefits:

- It gives another usage of the block edges in addition to sending messages
- It avoids the problem of priority queue exhausting that is one of restrictions of the original collective relational clustering algorithm
- It is more natural to the graph structure

4.7.2 Mathematical Foundation of the Communication

We can model our algorithm as a *Channel systems* [?]. Channel systems are popular for describing communication protocols. We give the intuitive definition of the Channel systems from [?]:

Intuitively, a channel system consists of n (data-dependent) processes P_1 through P_n . Each P_i is specified by a program graph PG_i which is extended with *communication actions*. Transitions of these program graphs are either the usual conditional transitions, or one of the communication actions with their respective intuitive meaning:

$c!v$ transmit the value v along channel c ,
 $c?x$ receive a message via channel c and assign it to variable x .

Channels are first-in, first-out buffers that may contain messages. Considering channel c as buffer, the communication action $c!v$ puts value v (at the rear of) the buffer whereas $c?x$ retrieves an element from (the front of) the buffer while assigning it to x . Let

$$Comm = \{ c!v, c?x \mid c \in Chan, v \in dom(c), x \in Var \text{ with } dom(x) \supseteq dom(c) \}$$

denote the set of communication actions where $Chan$ is a finite set of channels with typical element c .

A channel c has a (finite or infinite) *capacity* indicating the maximum number of messages it can store, and a type (or *domain*) specifying the type of messages that can be transmitted over c . Each channel c has a capacity $cap(c) \in \mathbb{N} \cup \{\infty\}$, and a domain $dom(c)$. For a channel c that can only transfer bits, $dom(c) = 0, 1$. If complete texts (of maximum length of 200, say) need to be transmitted over channel c , then another type of channel has to be used such that $dom(c) = \sum^{\leq 200}$, where $\sum$ is the alphabet that forms the basis of the texts, e.g., $\sum$ is the set of all letters and special characters used in German texts.

The capacity of a channel defines the size of the corresponding buffer, i.e., the number of messages not yet read that can be stored in the buffer. When $cap(c) \in \mathbb{N}$, c is a channel with finite capacity; $cap(c) = \infty$ indicates that c has an infinite capacity. Note that the special case $cap(c) = 0$ is permitted. In this case, channel c has no buffer. Communication via such a channel c corresponds to handshaking (simultaneous transmission and receipt, i.e., synchronous message passing) plus the exchange of some data. When $cap(c) > 0$, there is a "delay" between the transmission and the receipt of a message, i.e., sending and reading of the same message take place at different moments. This is called asynchronous message passing. Sending and reading a message from a channel with a nonzero capacity can never appear simultaneously. By means of channel systems, both synchronous and asynchronous message passing can thus be modeled.

We can map our algorithm to the Channel systems model such that the vertex corresponds to the process in the Channel systems model, Supersteps corresponds to the states of the program graph and the edge corresponds to the channel.

4.7.3 Complexity Analysis

We have two sets of graphs. The first set is the set of graphs built by adding the

4.7.4 Multi-type Entity Matching

Our framework supports multi-type entity matching. Different types of entities are connected by the relational edges in the relational graph. Then the references belong to each entity type is blocked such that similar references that belong to the same entity type are compared for matching. In the implementation we tag the type of entity. When building the relational graph

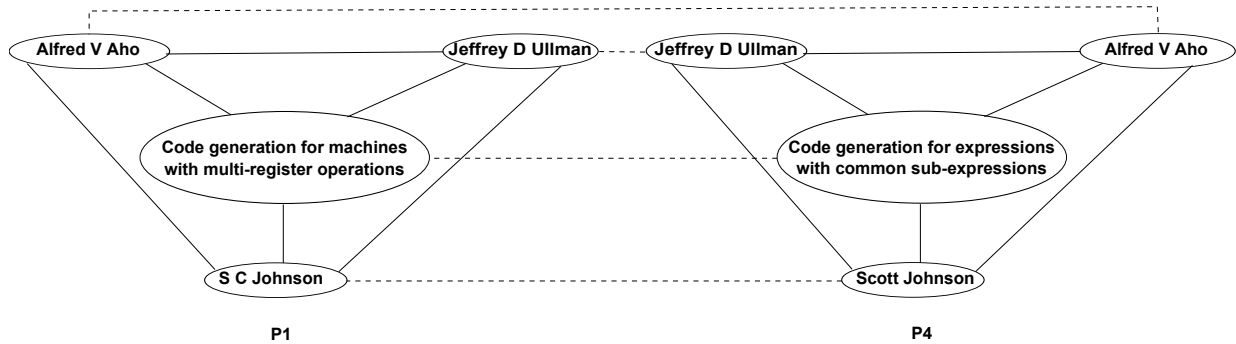


Figure 4.10: multi-type references (solid-lines: relation edge , dashed-line: block edge)

we connect instances of different entity types by relational edges to form a complete graph, i. e. every two vertices are connected by an edge.

Lets see how this is applied for our running example. Lets say that it is required to match the paper references in addition to matching the author references. Here we have multi-type entity matching where author references are matched and papers references are matched in the same task. All the references of different types are connected by the relational edges such that each two vertices are connected, this lead to having a complete graph. Figure 4.10 shows papers *P1* and *P4*. References to different types of entities (author and paper entity types) are connected by relational edges such that for each paper we have a complete graph (every twp vertices are connected). Then similar references that belong to the same entity type are blocked and block edges are used to connect the vertices corresponds to these references.

4.7.5 Canonicalization

Canonicalization is the construction of cluster representative with maximal information by combining information from its cluster elements. We implement this in our entity Info data structure.

4.7.6 Optimizing for the Cloud

Graph applications composed using the Pregel/BSP model encounter performance and scalability bottlenecks for applications that exhibit a log-normal distribution in messages exchanged during the supersteps when operating on small world graphs. this sharply ramp up the cumulative messages exchanged between supersteps as their traversals reach supernodes in the small world graph, and drain down with a long tail of supersteps whose total number typically corresponds to the graphs diameter. Issues with scalability for applications with variable message exchange patterns arise due to the corresponding changes in resources required over different supersteps that can lead to costly (money) over-provisioning or costly (time) underprovisioning of partition workers. On BSP frameworks messages must be buffered since they must be delivered by the end of a superstep but not drained (processed) until the next. This can be done either using (costly) disk access or by retaining them in memory, leading to memory pressure. We would like to maximize memory utilization for buffering messages without spilling over to virtual memory on disk, which may be even worse than disk-based buffering due to virtual memories random access (vs. sequential for disk-based buffering) I/O patterns.

Redekopp et al. proposed a vertex-scheduling model for Pregel/BSP, where computation for only a subset, or swath, of vertices is initiated at a time, though that computation may require action from all vertices in the graph as a traversal progresses. This approach allows resource utilization to be spread over time and shaped based on the swath size and the initiation interval between swaths. Clearly the swath size should be set as large as possible to provide good utilization but small enough to allow messages during peak supersteps to fully reside in physical memory.

4.8 Adaptive/Incremental Entity Matching

4.9 Graph-based Data Warehouse

Chapter 5

Implementation

5.1 Framework Architecture

We build our framework as a layer in top of the i5Cloud. The framework interact with the Openstack API to manage the instances that are running on the cloud. Figure 5.1 shows the architecture. The framework uses Apache Hama computing framework since it offers us both MapReduce and the BSP programming models. We use the MapReduce for the blocking and distribution phases, and we use the BSP based Pregel for the matching phase.

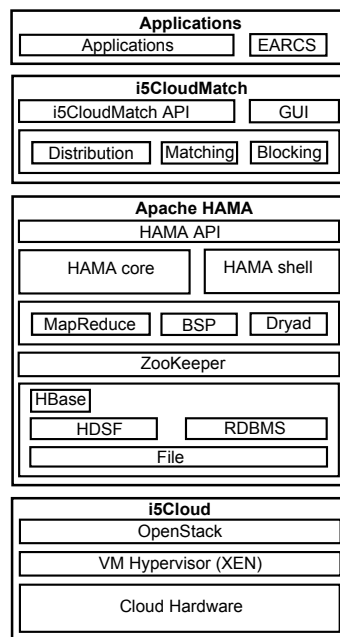


Figure 5.1: Framework Architecture

5.2 i5Cloud Test bed

i5Cloud is cloud computing platform hosted by the database and information systems chair in RWTH Aachen. The goal of i5Cloud is to provide infrastructure as a service (IaaS) and Platform as a service (Paas) for diverse applications and services ¹.

The first version i5Cloud 0.1 used Sun Enterprise server with Sun SPARC T5240 architecture. The current version i5Cloud 0.2 uses Sun server with Sun Fire x4450 architecture. This server system can expand to up to 24 processing cores, 256 GB of memory, and more than 2 TB of internal storage. i5Cloud 0.2 hosts the OpenStack² open-source project for cloud infrastructure management.

5.2.1 OpenStack

OpenStack is an open source software that provide infrastructure as service(IaaS). it consists of many components for managing the cloud resources (processing, storage, and networking resources). The components are Nova fabric controller, Swift salable redundant storage system, Cinder block storage, Quantum network management, Horizon dashboard, Keystone identification service and Glance image service.

5.3 Hama

Apache Hama is a pure BSP (Bulk Synchronous Parallel) computing framework on top of HDFS (Hadoop Distributed File System) for massive scientific computations such as matrix, graph and network algorithms. Hama focuses on general BSP computations, so it is not only for graphs. The overall architecture of Hama is illustrated in Figure 5.1 . Hama has a layered architecture consisting of three components: Hama Core for providing many primitives to matrix and graph computations, Hama Shell for interactive user console, and Hama API. The Hama Core component also determines the appropriate computation engine [SYK⁺10].

Hama supports three computation engines: MapReduce, BSP (Bulk Synchronous Parallel), and Dryad. The MapReduce is used for matrix computations, while BSP and Dryad are commonly used for graph computations. The main difference between BSP and Dryad is that BSP gives high performance with good data locality, while Dryad provides highly flexible computations with the fine control over the communication graph [SYK⁺10].

5.4 SimMetrics

SimMetrics is a similarity measures library. it provide many of the attribute based similarity measures that we need to implement like cosine, Jaccard, Levenstein, etc.

¹<http://dbis.rwth-aachen.de/cms/projects/i5cloud> accessed at 26.6.2013

²<http://www.openstack.org/>

Chapter 6

Evaluation

In this chapter we describe our evaluation plan. We give an overview of the quality measures, detailed evaluation plans and finally we give an overview of the data sets that we use in the evaluation.

6.1 Measuring Matching Quality

Evaluation of entity matching quality requires ground-truth data sets. By comparing the entity matcher decisions against the ground-truth data set, each matched record falls into one of the following four categories [Chr12]:

- *True positives(TP)*: These are the record pairs that have been decided as matches and that are true matches. These are the pairs where both records refer to the same entity.
- *False positives(FP)*: These are the record pairs that have been decided as matches, but they are not true matches. The two records in these pairs refer to two different entities. The matcher has made a wrong decision with these record pairs. These pairs are also known as false matches.
- *True negatives(TN)*: These are the record pairs that have been decided as non-matches, and they are true non-matches. The two records in pairs in this category do refer to two different real-world entities.
- *False negatives(FN)*: These are the record pairs that have been decided as non-matches, but they are actually true matches. The two records in these pairs refer to the same entity. The matcher has made a wrong decision with these record pairs. These pairs are also known as false non-matches.

Based on the number of true positives (TP), true negatives (TN), false positives (FP) and false negatives (FN), different quality measures can be calculated [Chr12].

6.1.0.1 Accuracy

This quality measure is calculated as:

$$acc = \frac{TP + TN}{TP + FP + TN + FN}$$

Accuracy is mainly useful for situations where the number of instances (record pairs in the case of data matching) are more or less the same for both the matches and the non-matches.

This accuracy measure is not suitable to properly assess matching quality since the no of matched records are rare in data matching and deduplication since the majority of record pairs corresponds to true non-matches (true negatives). As a result, the value of TN dominates the calculation of accuracy.

6.1.0.2 Precision

This is a measure commonly used in information retrieval to assess the quality of search results. It is calculated as:

$$prec = \frac{TP}{TP + FP}$$

Because precision does not include the number of true negatives, it does not suffer from the problem that the accuracy measure does. Precision calculates the proportion of how many of the decided matches (TP + FP) have been correctly decided as true matches (TP). It thus measures how precise a matcher is in deciding true matches.

6.1.0.3 Recall

This is a second measure that is commonly used in information retrieval. It is calculated as:

$$rec = \frac{TP}{TP + FN}$$

Similar to precision, because recall does not include the number of true negatives, this measure does not suffer from the problem that the accuracy measure does. It measures the proportion of true matches (TP + FN) that have been decided correctly (TP). It thus measures how many of the actual true matching record pairs have been correctly decided as matches.

6.1.0.4 F-measure

This measure, also known as *f-score* or *f1-score*, calculates the harmonic mean between precision and recall:

$$fmeas = 2 \times \frac{prec \times rec}{prec + rec}$$

The f-measure combines precision and recall and only has a high value if both precision and recall are high. Aiming to achieve a high f-measure requires to find the best compromise between precision and recall.

6.2 Measuring Framework Efficiency

Average Message Transferred

6.3 Evaluation Plan

We evaluate our framework by measuring its entity matching quality as well as its efficiency and scalability. We plan to evaluate the framework using three evaluation data sets.

- The first evaluation data sets is a small subset of the DBLP and CiteSeerX data sets where we already knows the ground truth of this subset. We modify the subset according to pre-defined scenarios to study our algorithms ability to handle these scenarios.
- The second evaluation data set is a small data set offered in Maryland university computer science web page¹. Its small subset of CiteSeerX with ground truth. it contains 1,504 machine learning documents with 2,892 author references to 1,165 author entities.
- The third evaluation data sets are the complete data sets of DBLP and CiteSeerX. With these data sets we evaluate the scalability and overall matching quality of our framework.

6.4 Test Data Sets

6.4.0.5 DBLP

DBLP (Digital Bibliography and Library Project) is online database containing computer science journals and conference and workshop proceedings. It started as small experimental Web server in 1993 at the university of Trier. it evolved from a small bibliography specialized to database systems and logic programming to digital library covering most subfields of computer science.

6.4.0.6 CiteSeerX

CiteSeerX is an autonomous citation indexing system that focuses on the literature in the fields of computer and information science. it crawl a large number of scientific publications from the web to extract their citation list to construct a database of papers linked by the citation relation. However there are often multiple references to the same paper, authors are not always correctly identified and citations are not always resolved.

¹<http://www.cs.umd.edu/projects/linqs/projects/er/index.html>

Chapter 7

Time Plan

What is the effect of small-world structure (small average diameter and high clustering) on the efficiency of our algorithm

Bibliography

- [BG04] Indrajit Bhattacharya and Lise Getoor. Iterative Record Linkage for Cleaning and Integration. In *Proceedings of the 9th ACM SIGMOD workshop on Research issues in data mining and knowledge discovery*, DMKD 04, pages 11–18. ACM, 2004.
- [BG06] Indrajit Bhattacharya and Lise Getoor. Entity Resolution in Graphs. In *Mining Graph Data*, pages 311–344. John Wiley & Sons, Inc, 2006.
- [BG07] Indrajit Bhattacharya and Lise Getoor. Collective Entity Resolution in Relational Data. *ACM Transactions on Knowledge Discovery from Data*, 1(1), 2007.
- [BGMG⁺] Omar Benjelloun, Hector Garcia-Molina, Heng Gong, Hideki Kawai, Tait E. Larson, David Menestrina, and Sutthipong Thavisomboon. D-Swoosh: A Family of Algorithms for Generic, Distributed Entity Resolution. In *27th International Conference on Distributed Computing Systems*, ICDCS ’07, pages 37–37.
- [BGMM⁺09] Omar Benjelloun, Hector Garcia-Molina, David Menestrina, Qi Su, Steven Euijong Whang, and Jennifer Widom. Swoosh: A Generic Approach to Entity Resolution. *The VLDB Journal*, 18(1):255–276, 2009.
- [BLGS06] Mustafa Bilgic, Louis Licamele, Lise Getoor, and Ben Shneiderman. D-Dupe: An Interactive Tool for Entity Resolution in Social Networks. In *Proceedings of the 13th international conference on Graph Drawing*, GD05, pages 505–507. Springer-Verlag, 2006.
- [BM03] Mikhail Bilenko and Raymond J. Mooney. Adaptive Duplicate Detection Using Learnable String Similarity Measures. In *Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining*, KDD 03, pages 39–48. ACM, 2003.
- [CCGK07] Surajit Chaudhuri, Bee-Chung Chen, Venkatesh Ganti, and Raghav Kaushik. Example-driven Design of Efficient Record Matching Queries. In *Proceedings of the 33rd international conference on Very large data bases*, VLDB 07, pages 327–338. VLDB Endowment, 2007.
- [CCH04] Peter Christen, Tim Churches, and Markus Hegland. Febrl – A Parallel Open Source Data Linkage System. In *Advances in Knowledge Discovery and Data*

Mining, volume 3056 of *Lecture Notes in Computer Science*, pages 638–647. Springer Berlin Heidelberg, 2004.

- [Chr08] Peter Christen. Febrl -: An Open Source Data Cleaning, Deduplication and Record Linkage System with a Graphical User Interface. In *Proceedings of the 14th ACM SIGKDD international conference on Knowledge discovery and data mining*, KDD 08, pages 1065–1068. ACM, 2008.
- [Chr12] Peter Christen. *Data matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*. Data-centric systems and applications. Springer, 2012.
- [CKM09] Zhaoqi Chen, Dmitri V. Kalashnikov, and Sharad Mehrotra. Exploiting Context Analysis for Combining Multiple Entity Resolution Systems. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of data*, SIGMOD 09, pages 207–218. ACM, 2009.
- [CRF03] William W. Cohen, Pradeep Ravikumar, and Stephen E. Fienberg. A Comparison of String Distance Metrics for Name-matching Tasks. In *Proceedings of Workshop on Information Integration on the Web (IIWeb-03)*, IJCAI-03, pages 73–78. 2003.
- [DBGH11] Guilherme Dal Bianco, Renata Galante, and Carlos A. Heuser. A Fast Approach for Parallel Deduplication on Multicore Processors. In *Proceedings of the 2011 ACM Symposium on Applied Computing*, SAC 11, pages 1027–1032. ACM, 2011.
- [DG08] Jeffrey Dean and Sanjay Ghemawat. MapReduce: Simplified Data Processing on Large Clusters. *Communications of the ACM (CACM)*, 51(1):107–113, 2008.
- [DHM05] Xin Dong, Alon Halevy, and Jayant Madhavan. Reference Reconciliation in Complex Information Spaces. In *Proceedings of the 2005 ACM SIGMOD international conference on Management of data*, SIGMOD 05, pages 85–96. ACM, 2005.
- [DN10] Uwe Draisbach and Felix Naumann. Dude: The Duplicate Detection Toolkit. In *Proceedings of the International Workshop on Quality in Databases (QDB)*, 2010.
- [EIV07] Ahmed K. Elmagarmid, Panagiotis G. Ipeirotis, and Vassilios S. Verykios. Duplicate Record Detection: A Survey. *IEEE Transactions on Knowledge and Data Engineering*, 19(1):1–16, 2007.
- [EVE02] Mohamed G. Elfeky, Vassilios S. Verykios, and Ahmed K. Elmagarmid. Tailor: A Record Linkage Tool Box. In *Proceedings of the 18th International Conference on Data Engineering*, ICDE 02, pages 17–28. IEEE Computer Society, 2002.

- [FS69] IvanP Fellegi and Alan B. Sunter. A Theory for Record Linkage. *Journal of the American Statistical Association*, 64(328):1183–1210, 1969.
- [GMT07] Aristides Gionis, Heikki Mannila, and Panayiotis Tsaparas. Clustering Aggregation. *ACM Transactions on Knowledge Discovery from Data*, 1(1), 2007.
- [HS95] Mauricio A. Hernandez and Salvatore J. Stolfo. The Merge/purge Problem for Large Databases. *Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data*, 24(2):127–138, 1995.
- [JIB10] Anja Jentzsch, Robert Isele, and Christian Bizer. Silk-Generating RDF Links while Publishing or Consuming Linked Data. In *Poster at the International Semantic Web Conference*, ISWC2010, 2010.
- [JLX⁺08] Pawel Jurczyk, James J. Lu, Li Xiong, Janet D. Cragan, and Adolfo Correa. FRIL: A Tool for Comparative Record Linkage. *Proceedings of AMIA Annual Symposium*, pages 440–444, 2008.
- [KGMB⁺06] Hideki Kawai, Hector Garcia-Molina, Omar Benjelloun, David Menestrina, Euijong Whang, and Heng Gong. P-Swoosh: Parallel Algorithm for Generic Entity Resolution. Technical Report 2006-19, Stanford InfoLab, 2006.
- [KIKK12] Tomasz Kajdanowicz, Wojciech Indyk, Przemyslaw Kazienko, and Jakub Kukul. Comparison of the Efficiency of MapReduce and Bulk Synchronous Parallel Approaches to Large Network Processing. *Proceedings of the 12th IEEE International Conference on Data Mining Workshops*, 0:218–225, 2012.
- [KKH⁺10] Toralf Kirsten, Lars Kolb, Michael Hartung, Anika Gross, Hanna Koepcke, and Erhard Rahm. Data Partitioning for Parallel Entity Matching. *Proceedings of the 8th International Workshop on Quality in Databases (QDB 2010)*, 2010.
- [KKTR11] Lars Kolb, Hanna Kpcke, Andreas Thor, and Erhard Rahm. Learning-based Entity Resolution with MapReduce. In *Proceedings of the third international workshop on Cloud data management*, CloudDB 11, pages 1–6. ACM, 2011.
- [KL07] Hung-sik Kim and Dongwon Lee. Parallel Linkage. In *Proceedings of the sixteenth ACM conference on Conference on information and knowledge management*, CIKM 07, pages 283–292. ACM, 2007.
- [KM06] Dmitri V. Kalashnikov and Sharad Mehrotra. Domain-independent Data Cleaning via Analysis of Entity-relationship Graph. *ACM Transactions on Database Systems*, 31(2):716–767, 2006.
- [KMPG09] Charalampos Konstantopoulos, Basilis Mamalis, Grammati Pantziou, and Damianos Gavalas. Efficient Parallel Text Retrieval Techniques on Bulk Synchronous Parallel (BSP)/Coarse Grained Multicomputers (CGM). *Journal of Supercomputing*, 48(3):286–318, 2009.

- [KR08] Hanna Koepcke and Erhard Rahm. Training Selection for Tuning Entity Matching. In *Proceedings of the International Workshop on Quality in Databases and Management of Uncertain Data*, pages 3–12, 2008.
- [KR10] Hanna Koepcke and Erhard Rahm. Frameworks for Entity Matching: A Comparison. *Data and Knowledge Engineering (DKE)*, 69(2):197–210, 2010.
- [KTR12] Lars Kolb, Andreas Thor, and Erhard Rahm. Multi-pass Sorted Neighborhood Blocking with MapReduce. *Computer Science - Research and Development*, 27(1):45–63, 2012.
- [LCW07] Luis Leitao, Pavel Calado, and Melanie Weis. Structure-based Inference of XML Similarity for Fuzzy Duplicate Detection. In *Proceedings of the sixteenth ACM conference on Conference on information and knowledge management, CIKM 07*, pages 293–302. ACM, 2007.
- [MAB⁺10] Grzegorz Malewicz, Matthew H. Austern, Aart J.C Bik, James C. Dehnert, Ilan Horn, Naty Leiser, and Grzegorz Czajkowski. Pregel: A System for Large-scale Graph Processing. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data*, SIGMOD 10, pages 135–146. ACM, 2010.
- [MNU00] Andrew McCallum, Kamal Nigam, and Lyle H. Ungar. Efficient Clustering of High-dimensional Data Sets with Application to Reference Matching. In *Proceedings of the sixth ACM SIGKDD international conference on Knowledge discovery and data mining*, KDD 00, pages 169–178. ACM, 2000.
- [NMMMBLP07] Jordi Nin, Victor Muntès-Mulero, Norbert Martínez-Bazan, and Josep-L Larriba-Pey. On the Use of Semantic Blocking Techniques for Data Cleansing and Integration. In *Proceedings of the 11th International Database Engineering and Applications Symposium, IDEAS 07*, pages 190–198. IEEE Computer Society, 2007.
- [RDG11] Vibhor Rastogi, Nilesh Dalvi, and Minos Garofalakis. Large-scale Collective Entity Matching. *Proceedings of the VLDB Endowment*, 4(4):208–218, 2011.
- [SB10] Murat Sariyar and Andreas Bog. The RecordLinkage Package: Detecting Errors in Data. *The R Journal*, 2/2:61–67, 2010.
- [SBB04] Rainer Schnell, Tobias Bachteler, and Stefan Bender. A Toolbox for Record Linkage. *Austrian Journal of Statistics*, 33(1-2):125–133, 2004.
- [SPY10] Jimeng Sun, Spiros Papadimitriou, and Rong Yan. Large-scale Data Mining: MapReduce and Beyond. In *The 16th ACM SIGKDD Conference on knowledge Discovery and Data Mining*, 2010.
- [SYK⁺10] Sangwon Seo, Edward J. Yoon, Jaehong Kim, Seongwook Jin, Jin-Soo Kim, and Seungryoul Maeng. HAMA: An Efficient Matrix Computation with the MapReduce Framework. In *Proceedings of the 2010 IEEE Second International Conference on Cloud Computing Technology and Science*, CLOUDCOM 10, pages 721–726. IEEE Computer Society, 2010.

- [Tal11] John R. Talburt. *Entity Resolution and Information Quality*. Morgan Kaufmann and Elsevier Science, 2011.
- [TKM02] Sheila Tejada, Craig A. Knoblock, and Steven Minton. Learning Domain-independent String Transformation Weights for High Accuracy Object Identification. In *Proceedings of the eighth ACM SIGKDD international conference on Knowledge discovery and data mining*, KDD 02, pages 350–359. ACM, 2002.
- [TR] Andreas Thor and Erhard Rahm. Moma - A Mapping-based Object Matching System. In *Third Biennial Conference on Innovative Data Systems Research*.
- [Val90] Leslie G. Valiant. A Bridging Model for Parallel Computation. *Communications of the ACM (CACM)*, 33(8):103–111, 1990.
- [Yan07] William Yancey. BigMatch: A Program for Extracting Probable Matches from a Large File. Technical report, US Census Bureau, 2007.
- [ZR05] Huimin Zhao and Sudha Ram. Entity Identification for Heterogeneous Database Integration: A Multiple Classifier System Approach and Empirical Evaluation. *Information Systems*, 30(2):119–132, 2005.